

Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention

Ali Hatamizadeh Yejin Choi Jan Kautz

{ahatamizadeh, yejinc, jkautz}@nvidia.com

Abstract: Linear attention replaces the unbounded cache of softmax attention with a fixed-size recurrent state, reducing sequence mixing to linear time and decoding to constant memory. The hard part is not just what to forget, but how to edit this compressed memory without scrambling existing associations. Delta-rule models subtract the current read before writing a new value, and Kimi Delta Attention (KDA) sharpens forgetting with channel-wise decay. But the active edit still uses a single scalar gate to control two different things, how much old content to erase on the key side and how much new content to commit on the value side. We introduce Gated DeltaNet-2, which generalizes both Gated DeltaNet and KDA by inheriting adaptive forgetting and channel-wise decay while addressing their shared limitation, the scalar tie between erasing and writing. Gated Delta Rule-2 separates these roles with a channel-wise erase gate b_t and a channel-wise write gate w_t , reducing to KDA when both gates collapse to the same scalar and to Gated DeltaNet when the decay also collapses. We derive a fast-weight update view, a chunkwise WY algorithm with channel-wise decay absorbed into asymmetric erase factors, and a gate-aware backward pass that preserves efficient parallel training. At 1.3B parameters trained on 100B FineWeb-Edu tokens, Gated DeltaNet-2 achieves the strongest overall results among Mamba-2, Gated DeltaNet, KDA, and Mamba-3 variants across language modeling, commonsense reasoning, and retrieval. Its advantage is most pronounced on long-context RULER needle-in-a-haystack benchmarks, where it improves the evaluated multi-key retrieval setting and remains strong in both recurrent and hybrid settings.

Code: <https://github.com/NVlabs/GatedDeltaNet-2>

1. Introduction

The Transformer architecture has become the dominant backbone for large language models because self-attention gives each token direct access to its history and maps naturally to parallel training on modern accelerators. Its cost, however, still grows quadratically with sequence length. This cost becomes a central obstacle for long-context training and high-throughput inference, where the model must repeatedly process histories that are much longer than the dimension of a single attention head.

Linear recurrent attention takes a different path. It replaces the explicit attention matrix with a fixed-size recurrent state, turning sequence mixing into a linear-time recurrence whose memory does not grow with context length [1]. The appeal is clear, but so is the constraint. The state is a compressed key-value memory, and long contexts force many associations to share the same finite space, making exact retrieval difficult [2, 3, 4, 5, 6, 7]. Recent work has improved this memory by giving the recurrence more control over what persists. Mamba-2 uses data-dependent decay to regulate the memory horizon [8]. DeltaNet replaces additive writes with the delta rule, enabling targeted overwrite of the association addressed by the current key [2, 9, 10]. Gated DeltaNet combines the delta rule with a learned decay gate, giving the state both global forgetting and targeted editing [11]. Kimi Delta Attention (KDA) refines the decay side with channel-wise forgetting over the key dimension [12]. In parallel, Mamba-3 advances the state-space route through exponential-trapezoidal discretization, complex-valued state transitions, and a multi-input, multi-output formulation for stronger and more efficient recurrence [13]. These advances have pushed recurrent linear models forward, while making the remaining bottleneck in delta-rule memory more visible. The active edit still uses one scalar gate to control both erasing old content and writing new content.

We propose *Gated DeltaNet-2*, a recurrent attention layer that decouples erase and write in the delta rule. The scalar tie is a modeling restriction because erasing and writing act on different axes of the state. Erasing is a key-side operation that decides which coordinates of the old read should be removed, while writing is a value-side operation that decides which coordinates of the incoming value should be committed. Gated DeltaNet-2 preserves KDA's channel-wise decay, but replaces the tied scalar delta gate with a channel-wise erase gate on the key axis and a channel-wise write gate on the value axis. The model can clear broad context through decay, remove selected stale associations through erase, and insert only the value channels that should persist through write. When the erase and write gates are tied to the same scalar, Gated DeltaNet-2 recovers KDA. If the decay is tied to a scalar as well, it recovers Gated DeltaNet.

This change preserves the efficient training path. By absorbing cumulative channel-wise decay into the rank-one erase factors, the recurrence admits a compact WY form with the same high-level chunkwise structure used by efficient delta-rule kernels [14, 15, 16, 17]. The main text gives the modeling equations and the chunkwise algorithm. Kernel-level details are deferred to the supplement.

Empirically, Gated DeltaNet-2 improves the recurrent attention frontier, with the clearest gains on long-context retrieval. On the RULER needle-in-a-haystack tasks in Table 3, it remains strong as context length grows and is especially effective on the evaluated multi-key case where a fixed-size state must separate competing associations. This advantage also appears in real-world recall, where Gated DeltaNet-2 gives the strongest overall retrieval profile in both recurrent and hybrid settings. Together with gains in language modeling, commonsense reasoning and in-context retrieval, these results suggest that decoupling the active memory edit directly targets the main pressure point of fixed-state recurrence, interference among many compressed associations.

2. Preliminary

2.1. Linear attention as a recurrent state

We work with one attention head and omit layer indices. Let $\mathbf{q}_t, \mathbf{k}_t \in \mathbb{R}^{d_k}$ and $\mathbf{v}_t \in \mathbb{R}^{d_v}$ denote the query, key, and value at position t . A recurrent linear attention layer stores a matrix state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ and reads it with the query,

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t. \quad (1)$$

This is the recurrent form of linear attention [1]. Expanding the recurrence over a length L sequence gives the familiar causal matrix form

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V}, \quad (2)$$

where $\mathbf{M}$ is the causal mask. The state has fixed size in L , and the parallel form replaces tokenwise recurrence with matrix multiplication. The limitation is equally direct. Every outer product is added to the state and none is removed, so old associations remain until they are overwritten indirectly by later superposition.

Chunkwise form Efficient linear recurrent layers use a chunkwise schedule during training [15, 16, 17]. Split the sequence into chunks of size C . For chunk n , let $\mathbf{Q}_{[n]}, \mathbf{K}_{[n]}, \mathbf{V}_{[n]}$ be the query, key, and value blocks, and let $\mathbf{S}_{[n]}$ be the state at the start of the chunk. Partial expansion gives

$$\mathbf{S}_{[n+1]} = \mathbf{S}_{[n]} + \mathbf{K}_{[n]}^\top \mathbf{V}_{[n]}, \quad \mathbf{O}_{[n]} = \mathbf{Q}_{[n]} \mathbf{S}_{[n]} + (\mathbf{Q}_{[n]} \mathbf{K}_{[n]}^\top \odot \mathbf{M}_C) \mathbf{V}_{[n]}. \quad (3)$$

The recurrence remains only across chunks, while all token interactions inside a chunk are expressed as dense matrix products. With a fixed C , this keeps linear complexity in sequence length and maps well to tensor cores.

2.2. Forgetting and overwriting

Mamba-2 adds a data-dependent scalar decay before each write [8],

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top, \quad \alpha_t \in (0, 1]. \quad (4)$$

The decay gives the model a global forgetting operation. If $\gamma_t = \prod_{i=1}^t \alpha_i$, then each earlier write is read at time t with factor γ_t/γ_i . This yields a decay-aware attention mask and preserves the chunkwise structure of Eq. 3.

DeltaNet instead gives the state an active edit operation [2, 9, 10]. Before writing $\mathbf{v}_t$, the model reads the value currently associated with $\mathbf{k}_t$ and subtracts it from the state. With a scalar step size $\beta_t \in [0, 1]$, the update is

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t (\mathbf{v}_t - \mathbf{S}_{t-1}^\top \mathbf{k}_t)^\top = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top. \quad (5)$$

When $\|\mathbf{k}_t\|_2 = 1$, the matrix $\mathbf{k}_t \mathbf{k}_t^\top$ is a projector, so $\beta_t = 1$ overwrites the association at key $\mathbf{k}_t$ and $\beta_t = 0$ leaves it unchanged. In the fast-weight view [18, 19], Eq. 5 is one online gradient step on the local regression loss $\frac{1}{2} \|\mathbf{S}^\top \mathbf{k}_t - \mathbf{v}_t\|_2^2$.

Gated DeltaNet combines these two operations [11],

$$\mathbf{S}_t = \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top. \quad (6)$$

The decay clears the state uniformly, while the delta rule edits a selected association. This is a useful division of labor, but both gates are scalar per head.

KDA refines the decay side by replacing the scalar α_t with a channel-wise vector $\alpha_t \in (0, 1]^{d_k}$ [12]. With $\mathbf{D}_t = \text{Diag}(\alpha_t)$, its update can be written as

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top. \quad (7)$$

KDA lets each key channel decay at its own rate and retains the efficient WY-based chunkwise algorithm of DeltaNet [10, 14]. Yet the active gate β_t is still a single scalar. It controls both how much old content is erased from the read direction and how much new value is written. Gated DeltaNet-2 starts from this remaining tie.

3. Gated DeltaNet-2

3.1. Decoupling erase and write

KDA refines Gated DeltaNet by making the decay channel-wise, but the scalar β_t in Eq. 7 still carries two decisions that need not agree. One decision lives on the key side and determines which coordinates of the current read should be erased. The other lives on the value side and determines which coordinates of the candidate value should be written. Treating both decisions as one scalar is a restriction of the update, not a requirement of the delta rule.

Gated DeltaNet-2 separates the two decisions through *Gated Delta Rule-2*. Let

$$\mathbf{e}_t = \mathbf{b}_t \odot \mathbf{k}_t, \quad \mathbf{z}_t = \mathbf{w}_t \odot \mathbf{v}_t, \quad (8)$$

where $\mathbf{b}_t \in [0, 1]^{d_k}$ is the erase gate and $\mathbf{w}_t \in [0, 1]^{d_v}$ is the write gate. The erase gate weights the key coordinates used to read old content, while the write gate weights the value coordinates being inserted. Let $\mathbf{D}_t = \text{Diag}(\alpha_t)$. Applying decay before the active edit gives

$$\bar{\mathbf{S}}_t = \mathbf{D}_t \mathbf{S}_{t-1}, \quad \mathbf{r}_t = \bar{\mathbf{S}}_t^\top \mathbf{e}_t, \quad \mathbf{S}_t = \bar{\mathbf{S}}_t + \mathbf{k}_t (\mathbf{z}_t - \mathbf{r}_t)^\top. \quad (9)$$

Equivalently,

$$\boxed{\mathbf{S}_t = (\mathbf{I} - \mathbf{k}_t (\mathbf{b}_t \odot \mathbf{k}_t)^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \odot \mathbf{v}_t)^\top} \quad (10)$$

We refer to Eq. 10 as *Gated Delta Rule-2*. The output is $\mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$. The left factor of the erase matrix remains $\mathbf{k}_t$, which preserves the write direction of the delta rule. The right factor becomes $\mathbf{b}_t \odot \mathbf{k}_t$, which makes the read direction channel selective. The write term becomes $\mathbf{k}_t \mathbf{z}_t^\top$, which makes the value update channel selective.

Gated Delta Rule-2 recovers KDA exactly when $\mathbf{b}_t = \beta_t \mathbf{1}_{d_k}$ and $\mathbf{w}_t = \beta_t \mathbf{1}_{d_v}$. It recovers Gated DeltaNet by further setting $\alpha_t = \alpha_t \mathbf{1}_{d_k}$. Thus the model preserves the known scalar-gated updates as tied subspaces, while learning outside those subspaces when erase and write require different channel structure.

The layer produces the two gates with independent projections of the token representation,

$$\mathbf{b}_t = \sigma(\mathbf{W}_b \mathbf{x}_t), \quad \mathbf{w}_t = \sigma(\mathbf{W}_w \mathbf{x}_t). \quad (11)$$

The log-decay follows the Gated DeltaNet parameterization,

$$\mathbf{g}_t = -\exp(\mathbf{a}) \odot \text{softplus}(\mathbf{W}_f \mathbf{x}_t + \boldsymbol{\delta}), \quad \alpha_t = \exp(\mathbf{g}_t). \quad (12)$$

In practice this decay activation is computed in fp32 before the kernel consumes it, which avoids precision loss in the cumulative log-decay. We also support the negative-eigenvalue variant of [20] by scaling only the erase gate to $[0, 2]^{d_k}$. The write gate remains in $[0, 1]^{d_v}$ because the spectral effect concerns the state transition, not the value magnitude.

3.2. Fast-weight update perspective

We can interpret Gated Delta Rule-2 as an online update of a fast-weight memory state [21]. The state $\mathbf{S}_t$ stores transient key-value associations. At each token, the model first forms a decayed state $\bar{\mathbf{S}}_t = \mathbf{D}_t \mathbf{S}_{t-1}$, reads the old content through the gated erase direction $\mathbf{e}_t$, and writes a correction toward the gated value target $\mathbf{z}_t$.

Table 1 | Fast-weight update view of DeltaNet, Mamba-2, Gated DeltaNet, KDA, Mamba-3, and Gated DeltaNet-2. All updates use the state orientation of this paper, where $\mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$. Mamba-2 and Mamba-3 add gated key-value correlation terms to a decayed state. DeltaNet, Gated DeltaNet, KDA, and Gated DeltaNet-2 instead write a delta residual, the target value minus the value currently read from memory.

Method	Local objective $L_t(\mathbf{S})$	State update
DeltaNet [10]	$\ \mathbf{S} - \mathbf{S}_{t-1}\ _F^2$ $- 2 \langle \mathbf{S}^\top \mathbf{k}_t, \beta_t (\mathbf{v}_t - \mathbf{S}_{t-1}^\top \mathbf{k}_t) \rangle$	$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1}$ $+ \beta_t \mathbf{k}_t \mathbf{v}_t^\top$
Mamba-2 [8]	$\ \mathbf{S} - \alpha_t \mathbf{S}_{t-1}\ _F^2$ $- 2 \langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{v}_t \rangle$	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$
Gated DeltaNet [11]	$\ \mathbf{S} - \alpha_t \mathbf{S}_{t-1}\ _F^2$ $- 2 \langle \mathbf{S}^\top \mathbf{k}_t, \beta_t (\mathbf{v}_t - (\alpha_t \mathbf{S}_{t-1})^\top \mathbf{k}_t) \rangle$	$\mathbf{S}_t = \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1}$ $+ \beta_t \mathbf{k}_t \mathbf{v}_t^\top$
KDA [12]	$\ \mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\ _F^2$ $- 2 \langle \mathbf{S}^\top \mathbf{k}_t, \beta_t (\mathbf{v}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{k}_t) \rangle$	$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1}$ $+ \beta_t \mathbf{k}_t \mathbf{v}_t^\top$
Mamba-3 [13]	$\ \mathbf{S} - \alpha_t \mathbf{S}_{t-1}\ _F^2$ $- 2 \langle \mathbf{S}^\top \tilde{\mathbf{k}}_{t-1}, \eta_t \mathbf{v}_{t-1} \rangle$ $- 2 \langle \mathbf{S}^\top \tilde{\mathbf{k}}_t, \zeta_t \mathbf{v}_t \rangle$	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \eta_t \tilde{\mathbf{k}}_{t-1} \mathbf{v}_{t-1}^\top$ $+ \zeta_t \tilde{\mathbf{k}}_t \mathbf{v}_t^\top$
Gated DeltaNet-2	$\ \mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\ _F^2$ $- 2 \langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{z}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{e}_t \rangle$	$\mathbf{S}_t = (\mathbf{I} - \mathbf{k}_t \mathbf{e}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1}$ $+ \mathbf{k}_t \mathbf{z}_t^\top$

More formally, Eq. 9 is the solution of the local online problem

$$\mathbf{S}_t = \arg \min_{\mathbf{S}} L_t(\mathbf{S}), \quad L_t(\mathbf{S}) = \|\mathbf{S} - \bar{\mathbf{S}}_t\|_F^2 - 2 \langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{z}_t - \bar{\mathbf{S}}_t^\top \mathbf{e}_t \rangle. \quad (13)$$

The first term keeps the new state close to the decayed memory. The second term applies an associative edit whose residual compares the gated write target $\mathbf{z}_t$ against the content read from $\bar{\mathbf{S}}_t$ along $\mathbf{e}_t$. Since

$$\nabla_{\mathbf{S}} L_t(\mathbf{S}) = 2(\mathbf{S} - \bar{\mathbf{S}}_t) - 2\mathbf{k}_t (\mathbf{z}_t - \bar{\mathbf{S}}_t^\top \mathbf{e}_t)^\top, \quad (14)$$

the minimizer is

$$\mathbf{S}_t = \bar{\mathbf{S}}_t + \mathbf{k}_t (\mathbf{z}_t - \bar{\mathbf{S}}_t^\top \mathbf{e}_t)^\top, \quad (15)$$

which is exactly Eq. 9.

Table 1 compares this view with Mamba-2, Gated DeltaNet, KDA, and Mamba-3. We write all updates in the state orientation used in this paper, where $\mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$. Normalizer terms, kernel maps, output gates, and value projection gates are omitted for readability.

For the Mamba-3 row, we use the SISO exponential-trapezoidal recurrence [13]. Let

$$\tilde{\mathbf{k}}_s = \mathbf{R}_{1:s}^\top \mathbf{k}_s, \quad \eta_t = (1 - \lambda_t) \Delta_t \alpha_t, \quad \zeta_t = \lambda_t \Delta_t. \quad (16)$$

Here $\mathbf{R}_{1:s}$ is the cumulative data-dependent rotation from the complex SSM view, and the previous-token term is omitted at the beginning of a sequence. The MIMO version replaces each rank-one write with a sum over the MIMO rank and leaves the same online form intact.

The comparison separates two families. Mamba-2 and Mamba-3 write correlations into a decayed state. Mamba-3 makes this write more expressive through the exponential-trapezoidal input rule and data-dependent rotations, but it does not subtract a current read from the state. Gated DeltaNet and KDA instead perform a residual delta edit. KDA changes the decay from scalar to channel-wise while keeping the scalar residual $\beta_t (\mathbf{v}_t - \bar{\mathbf{S}}_t^\top \mathbf{k}_t)$. Gated DeltaNet-2 changes the residual itself to

$$\mathbf{z}_t - \bar{\mathbf{S}}_t^\top \mathbf{e}_t = \mathbf{w}_t \odot \mathbf{v}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top (\mathbf{b}_t \odot \mathbf{k}_t), \quad (17)$$

which decouples the coordinates used to erase from the coordinates used to write.

3.3. Chunkwise parallel training

We now show that Gated Delta Rule-2 keeps the same chunkwise structure as KDA. Consider one chunk and suppress the chunk index. Let

$$\mathbf{G}_r = \sum_{i=1}^r \mathbf{g}_i, \quad \gamma_r = \exp(\mathbf{G}_r), \quad \gamma_0 = \mathbf{1}_{d_k}. \quad (18)$$

Define the decay-normalized state $\hat{\mathbf{S}}_r$ by $\mathbf{S}_r = \text{Diag}(\gamma_r) \hat{\mathbf{S}}_r$. With $\hat{\mathbf{S}}_0 = \mathbf{S}_{[n]}$, Eq. 10 becomes a pure asymmetric delta recurrence,

$$\hat{\mathbf{S}}_r = (\mathbf{I} - \bar{\mathbf{k}}_r \bar{\mathbf{e}}_r^\top) \hat{\mathbf{S}}_{r-1} + \bar{\mathbf{k}}_r \mathbf{z}_r^\top, \quad \bar{\mathbf{k}}_r = \gamma_r^{-1} \odot \mathbf{k}_r, \quad \bar{\mathbf{e}}_r = \gamma_r \odot \mathbf{e}_r. \quad (19)$$

This normalization is the key to the efficient form. The channel-wise decay is absorbed into the two factors of each rank-one erase, while the update remains a product of matrices of the form $\mathbf{I} - \bar{\mathbf{k}}_r \bar{\mathbf{e}}_r^\top$.

Let $\mathbf{B} \in \mathbb{R}^{C \times d_k}$ and $\mathbf{W} \in \mathbb{R}^{C \times d_v}$ contain rows $\mathbf{b}_r^\top$ and $\mathbf{w}_r^\top$, respectively. For compact matrix notation, let $\gamma \in \mathbb{R}^{C \times d_k}$ contain rows $\gamma_r^\top$. Let $\bar{\mathbf{K}} \in \mathbb{R}^{C \times d_k}$, $\bar{\mathbf{E}} \in \mathbb{R}^{C \times d_k}$, and $\mathbf{Z} \in \mathbb{R}^{C \times d_v}$ contain rows $\bar{\mathbf{k}}_r^\top$, $\bar{\mathbf{e}}_r^\top$, and $\mathbf{z}_r^\top$. Equivalently,

$$\bar{\mathbf{E}} = \gamma \odot (\mathbf{B} \odot \mathbf{K}), \quad \mathbf{Z} = \mathbf{W} \odot \mathbf{V}. \quad (20)$$

Define the strictly lower triangular matrix

$$\mathbf{T} = \text{tril}(\bar{\mathbf{E}} \bar{\mathbf{K}}^\top, -1), \quad \mathbf{A} = (\mathbf{I} + \mathbf{T})^{-1}. \quad (21)$$

The WY auxiliaries are

$$\mathbf{Y} = \mathbf{A} \bar{\mathbf{E}}, \quad \mathbf{U} = \mathbf{A} \mathbf{Z}. \quad (22)$$

Here $\mathbf{Y}$ is the erase-side auxiliary and $\mathbf{U}$ is the write-side auxiliary. Since $\mathbf{T}$ is triangular with zero diagonal, $\mathbf{A}$ is obtained by a small forward substitution inside each chunk.

The end-of-chunk state is then

$$\mathbf{S}_{[n+1]} = \text{Diag}(\gamma_C) \mathbf{S}_{[n]} + \mathbf{K}_{\text{tail}}^\top (\mathbf{U} - \mathbf{Y} \mathbf{S}_{[n]}), \quad (23)$$

where row r of $\mathbf{K}_{\text{tail}}$ is $(\gamma_C / \gamma_r) \odot \mathbf{k}_r$. The output block is

$$\mathbf{O}_{[n]} = \mathbf{Q}_\gamma \mathbf{S}_{[n]} + \mathbf{A}_{qk} (\mathbf{U} - \mathbf{Y} \mathbf{S}_{[n]}), \quad (24)$$

where row r of $\mathbf{Q}_\gamma$ is $\gamma_r \odot \mathbf{q}_r$ and

$$(\mathbf{A}_{qk})_{rs} = \mathbf{1}_{r \geq s} \mathbf{q}_r^\top \text{Diag}(\gamma_r / \gamma_s) \mathbf{k}_s. \quad (25)$$

Equations 23 and 24 have the same shape as the KDA chunk equations. The only difference is how $\mathbf{Y}$ and $\mathbf{U}$ are formed. The erase gate enters through row r of $\bar{\mathbf{E}}$ as $\gamma_r \odot (\mathbf{b}_r \odot \mathbf{k}_r)$. The write gate enters through row r of $\mathbf{Z}$ as $\mathbf{w}_r \odot \mathbf{v}_r$. The rest of the computation is a triangular solve and dense matrix multiplication over fixed-size chunks. We use the UT transform [22] and implement these equations with fused Triton kernels [23]. Kernel schedules and precision choices are deferred to the supplement.

3.4. Gate-aware backward

The backward pass follows the same decomposition as the forward. Gradients first flow through the output equation and the inter-chunk state recurrence, both of which operate only on $\mathbf{A}_{qk}$, $\mathbf{K}_{\text{tail}}$, $\mathbf{Y}$, and $\mathbf{U}$. The only new accounting is the vector-Jacobian product through Eq. 22 and Eq. 21.

For scalar-gated delta rules, a factor β_r can be moved outside the dot products that accumulate the gradient of $\mathbf{A}$. That shortcut breaks for Gated Delta Rule-2. The write side contains a different diagonal gate over value channels, and the erase side contains a different diagonal gate over key channels. Therefore the gate factors must be present

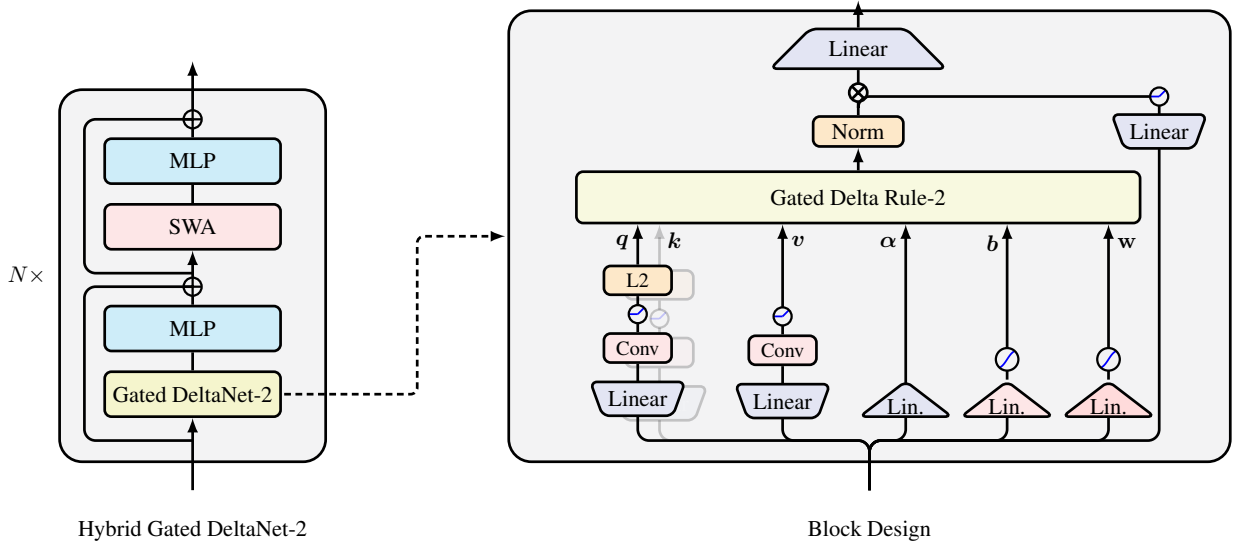


Figure 1 | Visualization of the hybrid architecture and block design of **Gated DeltaNet-2**. The *Hybrid Gated DeltaNet-2* model repeats a Gated DeltaNet-2 token mixer, an MLP, sliding-window attention (SWA), and another MLP. In the block design, query and key paths use linear projection, short convolution, SiLU, and L2 normalization. The value path uses linear projection, short convolution, and SiLU. The central recurrent operator is *Gated Delta Rule-2*. The decay branch produces α from the log-decay projection. The channel-wise erase gate b and channel-wise write gate w each use linear projection followed by sigmoid. The recurrent output is normalized, multiplied by a SiLU output gate, and passed through the output projection.

at the accumulation sites. Let $\mathbf{B}$ and $\mathbf{W}$ contain rows $b_r^\top$ and $w_r^\top$, respectively, and let γ denote the row-stacked cumulative-decay vectors. Then

$$d\mathbf{A} += d\mathbf{U} \mathbf{Z}^\top, \quad \mathbf{Z} = \mathbf{W} \odot \mathbf{V}, \quad (26)$$

$$d\mathbf{A} += d\mathbf{Y} \bar{\mathbf{E}}^\top, \quad \bar{\mathbf{E}} = \gamma \odot (\mathbf{B} \odot \mathbf{K}). \quad (27)$$

The inverse itself has the standard triangular vector-Jacobian product

$$d\mathbf{T} = -\text{tril}(\mathbf{A}^\top d\mathbf{A} \mathbf{A}^\top, -1). \quad (28)$$

From there, gradients to $\mathbf{B}$, $\mathbf{W}$, $\mathbf{K}$, $\mathbf{V}$, and the cumulative decay follow by ordinary elementwise products and reverse cumulative sums. This gate-aware accumulation is the main mathematical change required for training Gated Delta Rule-2. The remaining backward kernels retain the same matrix shapes as KDA and can reuse the same state and output vector-Jacobian product structure.

3.5. Block design and hybrid models

Gated DeltaNet-2 token mixer. Gated DeltaNet-2 is used as the recurrent token mixer in a standard Transformer-style block. Fig. 1 (right) shows its block design. For the Gated Delta Rule-2 in Eq. 10, $\{q_t, k_t, v_t\}$ are produced by linear projection, short causal convolution, and SiLU, with L2 normalization applied to q_t and k_t for stability. Separate branches produce the channel-wise decay α_t , erase gate b_t , and write gate w_t . The recurrent output is RMS-normalized, multiplied by a separate SiLU output gate, and projected back to the model dimension. Throughout the paper, g denotes the log-decay tensor in Eq. 12, not the output gate. With grouped value heads, q , k , the log-decay tensor g , and b are repeated across value-head groups, while v and w remain on the value-head axis.

Model families. We train both recurrent and hybrid models. The recurrent model stacks Gated DeltaNet-2 token mixers and MLPs under the standard residual block, isolating the fixed-state memory of Eq. 10. The hybrid model inserts Sliding-Window Attention (SWA) after the recurrent mixer, as shown in Fig. 1 (left). A repeated cell contains Gated DeltaNet-2, an MLP, SWA, and another MLP. Gated DeltaNet-2 compresses long histories into constant-size memory, while SWA handles exact local interactions such as short shifts, comparisons, and local retrieval. With a fixed window, the hybrid retains linear sequence scaling and a bounded attention cache, following the recurrent attention hybrid design pattern [24, 25].

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc ↑	OBQA acc ↑	SIQA acc ↑	BoolQ acc ↑	Avg. acc ↑
<i>Recurrent models</i>												
Mamba-2	16.79	12.38	45.24	<u>72.58</u>	55.51	55.33	70.68	35.26	31.00	40.63	60.19	51.82
Gated DeltaNet	16.40	11.89	49.62	72.31	<u>56.50</u>	<u>56.75</u>	68.81	35.15	30.20	40.53	58.78	52.07
KDA	16.81	11.68	<u>48.13</u>	72.09	55.75	<u>55.72</u>	70.83	35.92	30.40	<u>40.99</u>	60.67	52.28
Mamba-3 (SISO)	<u>16.30</u>	12.99	45.06	72.31	55.58	56.20	70.45	34.56	<u>31.00</u>	41.76	55.90	51.42
Mamba-3 (MIMO)	16.45	<u>11.66</u>	47.82	72.36	56.49	55.78	<u>72.38</u>	<u>38.07</u>	30.00	40.89	57.74	<u>52.39</u>
Gated DeltaNet-2	15.90	11.41	48.09	72.80	56.84	57.85	72.43	38.23	31.60	40.58	59.54	53.11
<i>Attention or hybrid models</i>												
Transformer	19.22	13.72	48.32	70.21	56.12	55.85	69.23	33.84	25.00	39.74	59.42	50.86
Mamba-2	17.46	11.29	48.05	71.47	57.52	56.17	70.50	34.73	29.80	40.35	59.31	51.99
Gated DeltaNet	16.00	10.82	48.71	70.06	57.50	56.83	70.41	35.15	30.60	40.97	60.00	52.25
KDA	16.01	10.66	49.21	71.06	56.89	<u>57.77</u>	<u>71.59</u>	35.07	30.00	40.53	<u>62.03</u>	52.68
Mamba-3 (SISO)	15.54	<u>10.65</u>	49.19	71.01	58.75	57.30	70.54	36.35	<u>32.00</u>	<u>41.20</u>	57.86	52.69
Mamba-3 (MIMO)	15.81	10.92	<u>49.82</u>	<u>71.98</u>	58.19	57.06	70.54	38.48	29.40	40.99	57.98	<u>52.72</u>
Gated DeltaNet-2	<u>15.62</u>	10.43	50.90	72.20	<u>58.46</u>	58.56	71.89	<u>36.69</u>	33.00	41.50	62.57	53.97

Table 2 | Performance comparison on language modeling and zero-shot common-sense reasoning. All accuracy values are reported as percentages. Avg. is computed over LAMBADA accuracy and the listed reasoning accuracies.

Model	S-NIAH-1				S-NIAH-2				S-NIAH-3			MK-NIAH-1		
	1K	2K	4K	8K	1K	2K	4K	8K	1K	2K	4K	1K	2K	4K
<i>Recurrent models</i>														
Mamba-2	100.0	100.0	97.0	55.8	99.6	<u>99.6</u>	62.6	21.0	59.2	38.6	14.4	29.0	21.2	21.4
Gated DeltaNet	<u>99.8</u>	100.0	100.0	97.6	100.0	100.0	87.2	32.0	89.8	54.2	60.6	58.0	37.0	27.8
KDA	100.0	100.0	<u>99.2</u>	70.6	100.0	100.0	<u>89.0</u>	30.6	77.4	63.2	26.2	54.0	<u>44.2</u>	<u>28.0</u>
Mamba-3 (SISO)	100.0	99.0	63.4	27.8	<u>99.8</u>	99.0	59.4	25.2	60.2	35.6	12.2	44.8	27.4	20.2
Mamba-3 (MIMO)	100.0	99.8	93.0	35.6	<u>99.8</u>	98.8	64.2	27.2	89.2	<u>72.4</u>	29.2	49.4	19.2	18.0
Gated DeltaNet-2	100.0	100.0	100.0	97.8	100.0	100.0	93.0	39.2	92.0	89.8	<u>31.8</u>	72.6	51.4	37.8
<i>Attention or hybrid models</i>														
Transformer	100.0	100.0	51.2	0.0	100.0	100.0	44.2	0.0	95.8	94.8	37.0	75.6	66.6	38.2
Mamba-2	100.0	100.0	<u>51.8</u>	25.4	100.0	99.6	52.4	25.8	97.8	86.8	48.0	82.0	58.6	39.0
Gated DeltaNet	100.0	100.0	47.2	22.4	100.0	<u>99.8</u>	57.3	25.6	94.8	91.2	47.2	91.0	78.4	44.8
KDA	100.0	100.0	<u>51.8</u>	<u>26.2</u>	100.0	100.0	56.0	23.0	97.2	93.4	51.6	<u>91.4</u>	<u>84.0</u>	40.4
Mamba-3 (SISO)	100.0	100.0	49.6	26.0	100.0	100.0	58.2	<u>27.8</u>	95.0	90.4	44.0	78.8	65.6	33.6
Mamba-3 (MIMO)	100.0	100.0	49.0	22.8	100.0	100.0	53.0	<u>27.8</u>	99.4	98.4	<u>54.2</u>	82.4	79.0	46.6
Gated DeltaNet-2	100.0	100.0	55.2	27.4	100.0	100.0	<u>57.9</u>	29.2	99.6	99.0	55.6	93.0	84.6	48.0

Table 3 | Accuracy on Single Needle-In-A-Haystack (S-NIAH) and Multi-Key Needle-In-A-Haystack (MK-NIAH) tasks from RULER. Best values within each model family and context length are bolded; second-best values are underlined.

4. Experiments

Setup We evaluate each recurrent family in two forms, a recurrent-only model and a hybrid model that pairs the same recurrent token mixer with sliding-window attention as described in Section 3.5. For Mamba-3, we include both SISO and MIMO variants and use MIMO rank $R = 4$ following [13]. All models are trained with the same recipe. Unless stated otherwise, each model has 1.3B parameters and is trained on 100B tokens from FineWeb-Edu [26]. We use AdamW with peak learning rate 4×10^{-4} , weight decay 0.1, gradient clipping at 1.0, cosine decay, a 1B-token warm-up, and a global batch size of 0.5M tokens. The training length is 4K tokens, and hybrid models use a 2K sliding-window attention size. Evaluation details are given in the appendix.

Language modeling and common-sense reasoning Table 2 reports WikiText and LAMBADA perplexity [27, 28], zero-shot LAMBADA accuracy, and the common-sense suite from PIQA through BoolQ [29, 30, 31, 32, 33, 34, 35]. Gated DeltaNet-2 achieves the best average in both recurrent and hybrid settings. Since recurrent state size is matched, the gain points to a stronger update rule rather than a larger memory. The trend persists with SWA, and the model is more balanced than Mamba-3 across perplexity, accuracy, and transfer.

In-context retrieval on synthetic data Table 3 reports S-NIAH and MK-NIAH from RULER [36], which test retention, interference control, high-entropy value storage, and multi-key discrimination under fixed-state memory. Gated DeltaNet-2 is strongest where memory editing matters most. In the recurrent setting, it leads the interference-heavy

Variant	Wiki. ppl ↓	LMB. ppl ↓	Common. avg ↑	S-NIAH-2 @4K ↑	S-NIAH-3 @2K ↑	MK-NIAH-1 @4K ↑	Recall avg ↑
<i>Channel structure</i>							
w-only, scalar b_t , channel w_t	16.55	11.62	52.45	90.6	71.4	30.6	28.92
b-only, channel b_t , scalar w_t	16.12	11.50	52.79	92.1	84.6	35.2	29.51
<i>Erase range</i>							
Gated DeltaNet-2, $b_t \in [0, 1]^{d_k}$	15.90	11.41	53.11	93.0	89.8	37.8	29.88
expanded $b_t \in [0, 2]^{d_k}$	15.95	11.44	53.04	93.1	89.4	37.6	29.81

Table 5 | Gate structure and erase range ablations in the recurrent-only setting.

S-NIAH-2 cases at 4K and 8K and all MK-NIAH-1 lengths. The hybrid model shows the same pattern, leading the long S-NIAH-1 cases, the 8K S-NIAH-2 case, all S-NIAH-3 lengths, and the longer MK-NIAH-1 settings. These gains match the design of Gated Delta Rule-2. The key-side erase gate b_t selectively protects or revises key channels, while the value-side write gate w_t controls which value channels enter the state. With SWA handling local evidence, this decoupled recurrent update preserves longer-range associations more effectively than a scalar delta gate.

In-context retrieval on real-world tasks

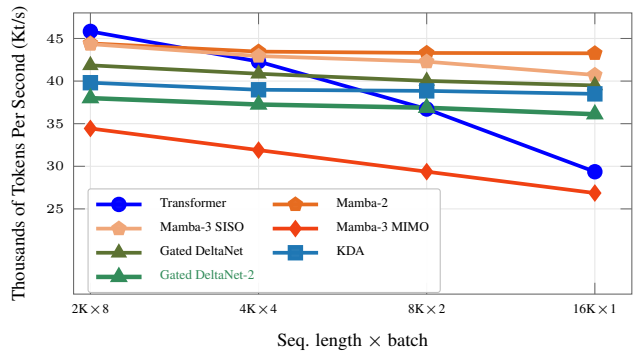
Table 4 reports recall-heavy real-world tasks from [37], spanning extraction, question answering, and distractor-rich evidence. These tasks are less controlled than synthetic NIAH but better reflect fixed-state memory under realistic context. Gated DeltaNet-2 achieves the best average in both recurrent and hybrid settings. Its recurrent gains are strongest on noisy association recovery, where selective erase and gated write are directly useful. The remaining NQ and DROP gaps point to formats that also need local evidence aggregation, which SWA supplies in the hybrid model.

Models	SWDE	SQD	FDA	TQA	NQ	DROP	Avg.
<i>Recurrent models</i>							
Mamba-2	17.24	32.38	14.53	58.35	18.91	19.60	26.84
Gated DeltaNet	17.90	32.67	18.52	59.60	20.16	19.69	28.09
Mamba-3 (SISO)	17.62	35.07	11.08	58.89	18.18	<u>21.32</u>	27.03
KDA	<u>22.49</u>	35.10	14.90	58.12	19.58	21.80	28.67
Mamba-3 (MIMO)	16.68	<u>36.65</u>	17.44	59.06	19.16	21.08	28.35
Gated DeltaNet-2	23.65	36.75	19.98	61.37	<u>19.64</u>	17.87	29.88
<i>Attention or hybrid models</i>							
Transformer	32.21	38.67	54.78	58.09	22.49	22.18	38.07
Mamba-2	34.67	40.74	52.31	60.13	25.91	24.68	39.74
Gated DeltaNet	33.18	42.28	50.86	<u>60.60</u>	25.78	21.95	39.11
Mamba-3 (SISO)	35.30	46.42	54.95	59.54	25.91	<u>23.96</u>	<u>41.01</u>
KDA	<u>39.83</u>	40.10	53.59	59.89	25.27	22.18	40.14
Mamba-3 (MIMO)	32.33	<u>44.70</u>	55.31	59.00	<u>26.26</u>	23.08	40.11
Gated DeltaNet-2	41.96	<u>44.70</u>	54.68	62.38	26.31	23.67	42.28

Table 4 | Accuracy on real-world retrieval tasks with input length truncated to 2K tokens. SQD denotes SQuAD. TQA denotes TriviaQA.

Gate structure and erase range ablations. Table 5 evaluates two aspects of the Gated Delta Rule-2 update, the channel structure of the erase and write gates, and the range of the erase gate. For the channel-structure ablations, we average either gate over its channel axis and broadcast the scalar back at runtime, while keeping the original projections unchanged. Thus the parameter count stays fixed and only channel-wise gate variation is removed. Both scalarized variants trail full Gated DeltaNet-2, showing that both gates use their channel degrees of freedom. The asymmetry is clear. Keeping channel structure only in b_t recovers most of the full model on language modeling and retrieval, whereas keeping it only in w_t recovers less. This matches Eq. 10, where b_t changes the key-side erase factor $k_t(b_t \odot k_t)^\top$, while w_t reweights the written value. Finally, expanding the erase range from $[0, 1]^{d_k}$ to $[0, 2]^{d_k}$ gives no consistent gain at this scale.

Throughput comparison. Fig. 2 reports single H100 training throughput for the hybrid 1.3B models under a fixed token budget. Gated DeltaNet-2 preserves the near-flat scaling profile of recurrent mixers as sequence length grows, dropping only mildly from 38.0 to 36.1 Kt/s, while the Transformer degrades sharply. Relative to KDA, the small gap reflects the added channel-wise erase and write gates. Thus Gated DeltaNet-2 retains practical training efficiency while paying a modest constant cost for finer memory control.

**Figure 2** | Training throughput on a H100 GPU.

5. Related Work

Efficient sequence models replace quadratic self-attention with recurrent or linear-time token mixers that maintain a fixed-size state. Early structured state-space and recurrent models used mostly data-independent transitions [16, 38, 39, 40], while Mamba and Mamba-2 introduced data-dependent selective dynamics and the SSD framework [8, 41]. Gated linear attention and related linear RNNs further improve memory control with learned decay gates [17, 42]. Delta-rule models take a complementary fast-weight view, where the recurrent state is updated by correcting the current read before writing the new value, improving associative memory over Hebbian-style accumulation [10, 43, 44, 45]. Gated DeltaNet adds adaptive forgetting to this update [11], and KDA strengthens it with channel-wise decay and an efficient chunkwise algorithm, but still uses a scalar β_t to control both erasing and writing [12]. Mamba-3 advances the SSM line instead, using exponential-trapezoidal discretization, complex-valued transitions implemented through data-dependent rotations, and a MIMO formulation for stronger modeling at efficient decoding latency [13]. Our work is complementary to these directions. Gated DeltaNet-2 keeps the delta-rule fast-weight structure of GDN and KDA, but replaces the tied scalar update strength with a channel-wise erase gate b_t and a channel-wise write gate w_t . This recovers KDA when both gates are tied to the same scalar, while allowing old content and new values to be controlled along different channel patterns.

6. Conclusion

We introduced Gated DeltaNet-2, a delta-rule recurrent attention layer that decouples the active memory edit into channel-wise erase and write decisions. The erase gate b_t selects which key-side coordinates of the decayed state are read and removed, while the write gate w_t selects which value-side coordinates are committed. This removes the scalar β_t tie in Gated DeltaNet and KDA, recovers both as special cases, and preserves efficient chunkwise training through a WY form with gate-aware kernels. Under matched 1.3B training, Gated DeltaNet-2 improves the recurrent and hybrid frontier across language modeling, commonsense reasoning, synthetic retrieval, and real-world recall, while adding only a small constant throughput overhead. Ablations show that both gates contribute, with the erase gate accounting for most of the gain.

A. Chunkwise derivation for Gated DeltaNet-2

This appendix gives the exact chunkwise form used in Section 3.3. We work inside one chunk of length C and write $\mathbf{S}_0$ for the state at the start of the chunk. All vectors are for a single head. In particular, $\mathbf{k}_r, \mathbf{e}_r, \mathbf{q}_r, \mathbf{b}_r, \boldsymbol{\alpha}_r, \boldsymbol{\gamma}_r \in \mathbb{R}^{d_k}$, $\mathbf{v}_r, \mathbf{z}_r, \mathbf{o}_r, \mathbf{w}_r \in \mathbb{R}^{d_v}$, and $\mathbf{S}_r \in \mathbb{R}^{d_k \times d_v}$. The chunk index is suppressed.

The Gated DeltaNet-2 recurrence is

$$\mathbf{S}_r = (\mathbf{I} - \mathbf{k}_r \mathbf{e}_r^\top) \text{Diag}(\boldsymbol{\alpha}_r) \mathbf{S}_{r-1} + \mathbf{k}_r \mathbf{z}_r^\top, \quad \mathbf{e}_r = \mathbf{b}_r \odot \mathbf{k}_r, \quad \mathbf{z}_r = \mathbf{w}_r \odot \mathbf{v}_r. \quad (29)$$

Let

$$\mathbf{G}_r = \sum_{i=1}^r \mathbf{g}_i, \quad \boldsymbol{\gamma}_r = \exp(\mathbf{G}_r), \quad \boldsymbol{\gamma}_0 = \mathbf{1}_{d_k}, \quad \boldsymbol{\alpha}_r = \exp(\mathbf{g}_r). \quad (30)$$

All exponentials, products, and ratios involving $\boldsymbol{\gamma}$ are elementwise over the key channel axis.

A.1. Decay-normalized recurrence

Define a normalized state $\hat{\mathbf{S}}_r$ by

$$\mathbf{S}_r = \text{Diag}(\boldsymbol{\gamma}_r) \hat{\mathbf{S}}_r. \quad (31)$$

Because $\boldsymbol{\gamma}_0 = \mathbf{1}_{d_k}$, the normalized initial state is also $\hat{\mathbf{S}}_0 = \mathbf{S}_0$. Substituting Eq. 31 into Eq. 29 and using $\boldsymbol{\gamma}_r = \boldsymbol{\alpha}_r \odot \boldsymbol{\gamma}_{r-1}$ gives

$$\hat{\mathbf{S}}_r = (\mathbf{I} - \bar{\mathbf{k}}_r \bar{\mathbf{e}}_r^\top) \hat{\mathbf{S}}_{r-1} + \bar{\mathbf{k}}_r \bar{\mathbf{z}}_r^\top, \quad \bar{\mathbf{k}}_r = \boldsymbol{\gamma}_r^{-1} \odot \mathbf{k}_r, \quad \bar{\mathbf{e}}_r = \boldsymbol{\gamma}_r \odot \mathbf{e}_r. \quad (32)$$

The channel-wise decay has disappeared from the recurrence. It is now carried by the left and right factors of each rank-one edit.

Let $\mathbf{K}, \mathbf{V}, \mathbf{B}$, and $\mathbf{W}$ contain rows $\mathbf{k}_r^\top, \mathbf{v}_r^\top, \mathbf{b}_r^\top$, and $\mathbf{w}_r^\top$, respectively. For compact matrix notation, let $\boldsymbol{\gamma}$ contain rows $\boldsymbol{\gamma}_r^\top$. Let $\bar{\mathbf{K}}, \bar{\mathbf{E}}$, and $\mathbf{Z}$ contain rows $\bar{\mathbf{k}}_r^\top, \bar{\mathbf{e}}_r^\top$, and $\mathbf{z}_r^\top$. Equivalently,

$$\bar{\mathbf{K}} = \boldsymbol{\gamma}^{-1} \odot \mathbf{K}, \quad \bar{\mathbf{E}} = \boldsymbol{\gamma} \odot (\mathbf{B} \odot \mathbf{K}), \quad \mathbf{Z} = \mathbf{W} \odot \mathbf{V}. \quad (33)$$

Define

$$\mathbf{T} = \text{tril}(\bar{\mathbf{E}} \bar{\mathbf{K}}^\top, -1), \quad \mathbf{A} = (\mathbf{I} + \mathbf{T})^{-1}, \quad \mathbf{Y} = \mathbf{A} \bar{\mathbf{E}}, \quad \mathbf{U} = \mathbf{A} \mathbf{Z}. \quad (34)$$

Since $\mathbf{T}$ is strictly lower triangular, $\mathbf{A}$ is lower triangular with unit diagonal and is obtained by forward substitution.

A.2. Compact state formula

Define

$$\mathbf{R} = \mathbf{U} - \mathbf{Y} \mathbf{S}_0. \quad (35)$$

Let row r of $\mathbf{R}$ be $\boldsymbol{\rho}_r^\top$, where $\boldsymbol{\rho}_r \in \mathbb{R}^{d_v}$. Then the normalized state after any prefix of the chunk is

$$\hat{\mathbf{S}}_r = \mathbf{S}_0 + \bar{\mathbf{K}}_{\leq r}^\top \mathbf{R}_{\leq r}, \quad (36)$$

where $\bar{\mathbf{K}}_{\leq r}$ and $\mathbf{R}_{\leq r}$ denote the first r rows.

To prove Eq. 36, write the rank-one increment at step r as $\bar{\mathbf{k}}_r \boldsymbol{\rho}_r^\top$. From Eq. 32, the residual row is

$$\boldsymbol{\rho}_r^\top = \mathbf{z}_r^\top - \bar{\mathbf{e}}_r^\top \hat{\mathbf{S}}_{r-1}. \quad (37)$$

Equivalently, in column-vector form, $\boldsymbol{\rho}_r = \mathbf{z}_r - \hat{\mathbf{S}}_{r-1}^\top \bar{\mathbf{e}}_r$. Using the induction hypothesis $\hat{\mathbf{S}}_{r-1} = \mathbf{S}_0 + \sum_{s < r} \bar{\mathbf{k}}_s \boldsymbol{\rho}_s^\top$ gives

$$\boldsymbol{\rho}_r^\top = \mathbf{z}_r^\top - \bar{\mathbf{e}}_r^\top \mathbf{S}_0 - \sum_{s < r} \bar{\mathbf{e}}_r^\top \bar{\mathbf{k}}_s \boldsymbol{\rho}_s^\top. \quad (38)$$

Since $\mathbf{T}_{rs} = \bar{\mathbf{e}}_r^\top \bar{\mathbf{k}}_s$ for $s < r$ and $\mathbf{T}_{rs} = 0$ otherwise, stacking these residual rows over the chunk yields

$$(\mathbf{I} + \mathbf{T})\mathbf{R} = \mathbf{Z} - \bar{\mathbf{E}}\mathbf{S}_0. \quad (39)$$

Multiplying by $\mathbf{A}$ gives $\mathbf{R} = \mathbf{AZ} - \mathbf{A}\bar{\mathbf{E}}\mathbf{S}_0 = \mathbf{U} - \mathbf{Y}\mathbf{S}_0$, which is Eq. 35. Substituting the resulting increments into the normalized recurrence proves Eq. 36.

Multiplying Eq. 36 by $\text{Diag}(\gamma_C)$ gives the end-of-chunk state

$$\mathbf{S}_C = \text{Diag}(\gamma_C)\mathbf{S}_0 + \mathbf{K}_{\text{tail}}^\top (\mathbf{U} - \mathbf{Y}\mathbf{S}_0), \quad (40)$$

where row r of $\mathbf{K}_{\text{tail}}$ is

$$(\mathbf{K}_{\text{tail}})_{r,:} = ((\gamma_C/\gamma_r) \odot \mathbf{k}_r)^\top. \quad (41)$$

This is Eq. 23 in the main text.

A.3. Compact output formula

The output at token r is the column vector $\mathbf{o}_r = \mathbf{S}_r^\top \mathbf{q}_r$. It is convenient to write the corresponding row vector. Using Eq. 36,

$$\mathbf{o}_r^\top = (\gamma_r \odot \mathbf{q}_r)^\top \mathbf{S}_0 + \sum_{s \leq r} [\mathbf{q}_r^\top \text{Diag}(\gamma_r/\gamma_s) \mathbf{k}_s] \boldsymbol{\rho}_s^\top. \quad (42)$$

Define $\mathbf{Q}_\gamma$ by row $(\mathbf{Q}_\gamma)_{r,:} = (\gamma_r \odot \mathbf{q}_r)^\top$ and define the causal score matrix

$$(\mathbf{A}_{qk})_{rs} = \mathbf{1}_{r \geq s} \mathbf{q}_r^\top \text{Diag}(\gamma_r/\gamma_s) \mathbf{k}_s. \quad (43)$$

Let $\mathbf{O}$ contain rows $\mathbf{o}_r^\top$. Stacking Eq. 42 over the chunk gives

$$\mathbf{O} = \mathbf{Q}_\gamma \mathbf{S}_0 + \mathbf{A}_{qk} (\mathbf{U} - \mathbf{Y}\mathbf{S}_0), \quad (44)$$

which is Eq. 24 in the main text.

A.4. Row recurrences

The matrices $\mathbf{Y}$ and $\mathbf{U}$ can also be written row by row. Let row r of $\mathbf{Y}$ be $\mathbf{y}_r^\top$ and row r of $\mathbf{U}$ be $\mathbf{u}_r^\top$. Since $(\mathbf{I} + \mathbf{T})\mathbf{Y} = \bar{\mathbf{E}}$ and $(\mathbf{I} + \mathbf{T})\mathbf{U} = \mathbf{Z}$,

$$\mathbf{y}_r^\top = \bar{\mathbf{e}}_r^\top - \sum_{s < r} \bar{\mathbf{e}}_r^\top \bar{\mathbf{k}}_s \mathbf{y}_s^\top, \quad (45)$$

$$\mathbf{u}_r^\top = \mathbf{z}_r^\top - \sum_{s < r} \bar{\mathbf{e}}_r^\top \bar{\mathbf{k}}_s \mathbf{u}_s^\top. \quad (46)$$

Both auxiliaries solve the same lower triangular system with different right-hand sides. This is why the same $\mathbf{WY}$ inverse can be shared by the erase-side and write-side computations.

A.5. Tied-gate reductions

If $\mathbf{b}_r = \beta_r \mathbf{1}_{d_k}$ and $\mathbf{w}_r = \beta_r \mathbf{1}_{d_v}$, then $\mathbf{e}_r = \beta_r \mathbf{k}_r$ and $\mathbf{z}_r = \beta_r \mathbf{v}_r$. Equation 29 becomes the KDA update. If the decay is also tied as $\alpha_r = \alpha_r \mathbf{1}_{d_k}$, the recurrence becomes Gated DeltaNet. Thus KDA and Gated DeltaNet are recovered by tying the channel gates rather than by changing the algorithm.

The same reduction holds for the chunkwise form. Under the KDA tying, the definitions in Eq. 33 give

$$\bar{\mathbf{e}}_r = \gamma_r \odot (\beta_r \mathbf{k}_r) = \beta_r (\gamma_r \odot \mathbf{k}_r) = \beta_r (\gamma_r \odot \gamma_r) \odot \bar{\mathbf{k}}_r, \quad \mathbf{z}_r = \beta_r \mathbf{v}_r. \quad (47)$$

Thus $\mathbf{Z}$ becomes a scalar row scaling of $\mathbf{V}$, while $\bar{\mathbf{E}}$ becomes the KDA decay-normalized erase factor. In general, $\bar{\mathbf{E}}$ is not a scalar row scaling of $\bar{\mathbf{K}}$ when the decay is channel-wise, since each key channel carries its own factor from γ_r .

Equations 34, 40, and 44 therefore reduce to the KDA chunk equations after substituting the tied factors in Eq. 47. If the decay is further tied as in Gated DeltaNet, then $\gamma_r = \gamma_r \mathbf{1}_{d_k}$ and the erase factor simplifies to $\bar{e}_r = \beta_r \gamma_r^2 \bar{\mathbf{k}}_r$, recovering the scalar-decay chunkwise form.

At the gradient level, if a thin wrapper sets $\mathbf{b}_r = \beta_r \mathbf{1}_{d_k}$ and $\mathbf{w}_r = \beta_r \mathbf{1}_{d_v}$, the scalar gradient is

$$\frac{\partial \mathcal{L}}{\partial \beta_r} = \left\langle \frac{\partial \mathcal{L}}{\partial \mathbf{b}_r}, \mathbf{1}_{d_k} \right\rangle + \left\langle \frac{\partial \mathcal{L}}{\partial \mathbf{w}_r}, \mathbf{1}_{d_v} \right\rangle. \quad (48)$$

B. Backward derivation

We derive the vector-Jacobian products for one chunk using the notation of Appendix A. Let upstream gradients be $d\mathbf{O}$ for the chunk output and $d\mathbf{S}_C$ for the end-of-chunk state. The forward equations are

$$\mathbf{R} = \mathbf{U} - \mathbf{Y}\mathbf{S}_0, \quad (49)$$

$$\mathbf{O} = \mathbf{Q}_\gamma \mathbf{S}_0 + \mathbf{A}_{qk} \mathbf{R}, \quad (50)$$

$$\mathbf{S}_C = \text{Diag}(\gamma_C) \mathbf{S}_0 + \mathbf{K}_{\text{tail}}^\top \mathbf{R}, \quad (51)$$

$$\mathbf{U} = \mathbf{A}\mathbf{Z}, \quad \mathbf{Y} = \mathbf{A}\bar{\mathbf{E}}, \quad \mathbf{A} = (\mathbf{I} + \mathbf{T})^{-1}, \quad \mathbf{T} = \text{tril}(\bar{\mathbf{E}}\bar{\mathbf{K}}^\top, -1). \quad (52)$$

B.1. Output and state paths

From Eq. 50,

$$d\mathbf{A}_{qk} += d\mathbf{O} \mathbf{R}^\top, \quad (53)$$

$$d\mathbf{R} += \mathbf{A}_{qk}^\top d\mathbf{O}, \quad (54)$$

$$d\mathbf{Q}_\gamma += d\mathbf{O} \mathbf{S}_0^\top, \quad (55)$$

$$d\mathbf{S}_0 += \mathbf{Q}_\gamma^\top d\mathbf{O}. \quad (56)$$

The causal mask is applied to $d\mathbf{A}_{qk}$.

From Eq. 51,

$$d\mathbf{R} += \mathbf{K}_{\text{tail}} d\mathbf{S}_C, \quad (57)$$

$$d\mathbf{K}_{\text{tail}} += \mathbf{R} d\mathbf{S}_C^\top, \quad (58)$$

$$d\mathbf{S}_0 += \text{Diag}(\gamma_C) d\mathbf{S}_C, \quad (59)$$

$$d\gamma_C += \text{rowsum}(d\mathbf{S}_C \odot \mathbf{S}_0). \quad (60)$$

Here rowsum sums over the value dimension and returns a d_k -dimensional vector.

The residual relation in Eq. 49 gives

$$d\mathbf{U} += d\mathbf{R}, \quad (61)$$

$$d\mathbf{Y} += -d\mathbf{R} \mathbf{S}_0^\top, \quad (62)$$

$$d\mathbf{S}_0 += -\mathbf{Y}^\top d\mathbf{R}. \quad (63)$$

B.2. Gate-aware WY inverse path

The two auxiliary products yield

$$d\mathbf{A} += d\mathbf{U} \mathbf{Z}^\top, \quad d\mathbf{Z} += \mathbf{A}^\top d\mathbf{U}, \quad (64)$$

$$d\mathbf{A} += d\mathbf{Y} \bar{\mathbf{E}}^\top, \quad d\bar{\mathbf{E}} += \mathbf{A}^\top d\mathbf{Y}. \quad (65)$$

Equations 64 and 65 are the gate-aware accumulation emphasized in Section 3.4. Since $\mathbf{Z} = \mathbf{W} \odot \mathbf{V}$ and $\bar{\mathbf{E}} = \gamma \odot (\mathbf{B} \odot \mathbf{K})$, the gates must appear inside the products that accumulate $d\mathbf{A}$. A scalar post-scale is correct only in the tied-gate case.

For the inverse,

$$d\mathbf{T} = -\text{tril}(\mathbf{A}^\top d\mathbf{A}\mathbf{A}^\top, -1). \quad (66)$$

The construction of $\mathbf{T}$ gives

$$d\bar{\mathbf{E}} += d\mathbf{T}\bar{\mathbf{K}}, \quad (67)$$

$$d\bar{\mathbf{K}} += d\mathbf{T}^\top \bar{\mathbf{E}}. \quad (68)$$

Only the strictly lower triangular part of $d\mathbf{T}$ is used.

B.3. Scores and tail keys

The score matrix satisfies $\mathbf{A}_{qk} = \text{tril}(\mathbf{Q}_\gamma \bar{\mathbf{K}}^\top)$. Therefore

$$d\mathbf{Q}_\gamma += d\mathbf{A}_{qk} \bar{\mathbf{K}}, \quad (69)$$

$$d\bar{\mathbf{K}} += d\mathbf{A}_{qk}^\top \mathbf{Q}_\gamma. \quad (70)$$

The tail key matrix satisfies $(\mathbf{K}_{\text{tail}})_r = \gamma_C \odot \bar{\mathbf{k}}_r$. Hence

$$d\bar{\mathbf{K}} += d\mathbf{K}_{\text{tail}} \odot \gamma_C, \quad (71)$$

$$d\gamma_C += \sum_{r=1}^C d(\mathbf{K}_{\text{tail}})_r \odot \bar{\mathbf{k}}_r. \quad (72)$$

B.4. Elementwise gates and cumulative decay

The write-side relation $\mathbf{Z} = \mathbf{W} \odot \mathbf{V}$ gives

$$d\mathbf{W} += d\mathbf{Z} \odot \mathbf{V}, \quad d\mathbf{V} += d\mathbf{Z} \odot \mathbf{W}. \quad (73)$$

The erase-side relation $\bar{\mathbf{E}} = \gamma \odot (\mathbf{B} \odot \mathbf{K})$ gives

$$d\mathbf{B} += d\bar{\mathbf{E}} \odot \gamma \odot \mathbf{K}, \quad (74)$$

$$d\mathbf{K} += d\bar{\mathbf{E}} \odot \gamma \odot \mathbf{B}, \quad (75)$$

$$d\gamma += d\bar{\mathbf{E}} \odot \mathbf{B} \odot \mathbf{K}. \quad (76)$$

The normalized keys and queries are

$$\bar{\mathbf{K}} = \gamma^{-1} \odot \mathbf{K}, \quad \mathbf{Q}_\gamma = \gamma \odot \mathbf{Q}. \quad (77)$$

Their vector-Jacobian products are

$$d\mathbf{K} += d\bar{\mathbf{K}} \odot \gamma^{-1}, \quad (78)$$

$$d\gamma += -d\bar{\mathbf{K}} \odot \mathbf{K} \odot \gamma^{-2}, \quad (79)$$

$$d\mathbf{Q} += d\mathbf{Q}_\gamma \odot \gamma, \quad (80)$$

$$d\gamma += d\mathbf{Q}_\gamma \odot \mathbf{Q}. \quad (81)$$

Finally, $\gamma_r = \exp(\mathbf{G}_r)$ and $\mathbf{G}_r = \sum_{i \leq r} \mathbf{g}_i$. Therefore

$$d\mathbf{G}_r = d\gamma_r \odot \gamma_r, \quad d\mathbf{g}_i = \sum_{r \geq i} d\mathbf{G}_r. \quad (82)$$

In implementation this is a reverse cumulative sum over the chunk.

B.5. Why scalar post-scaling is invalid

In KDA, a scalar β_r multiplies both the value right hand side and the erase right hand side. For one row, the contribution to $d\mathbf{A}$ from the write auxiliary can be factored as

$$d\mathbf{u}_r (\beta_s \mathbf{v}_s)^\top = \beta_s d\mathbf{u}_r \mathbf{v}_s^\top. \quad (83)$$

The scalar factor can be applied after the dot product. Gated DeltaNet-2 replaces $\beta_s \mathbf{v}_s$ by $\mathbf{w}_s \odot \mathbf{v}_s$. Since $\mathbf{w}_s$ is a different diagonal operator for every row, there is no row scalar or column scalar that can recover

$$d\mathbf{u}_r (\mathbf{w}_s \odot \mathbf{v}_s)^\top \quad (84)$$

from $d\mathbf{u}_r \mathbf{v}_s^\top$. The erase side has the same issue with $\mathbf{b}_s \odot \mathbf{k}_s$. The gates must be baked into the dot products in Eq. 64 and Eq. 65.

C. Layer and kernel implementation

This appendix records the implementation choices needed to reproduce Gated DeltaNet-2. The main text keeps the Triton details brief. Here we describe the computation at the level of kernels and tensor shapes.

C.1. Layer parameterization

The layer computes short-convolutional projections for $\mathbf{q}$, $\mathbf{k}$, and $\mathbf{v}$, followed by head reshaping. The erase and write gates are produced by independent projections,

$$\mathbf{b} = \sigma(\text{Proj}_b(\mathbf{x})), \quad \mathbf{w} = \sigma(\text{Proj}_w(\mathbf{x})). \quad (85)$$

The erase projection has shape $d_{\text{model}} \rightarrow Hd_k$. The write projection has shape $d_{\text{model}} \rightarrow H_v d_v$. If grouped value attention is used with $H_v > H$, the key-side tensors $\mathbf{q}$, $\mathbf{k}$, the log-decay tensor $\mathbf{g}$, and $\mathbf{b}$ are repeated across the value-head group, while $\mathbf{v}$ and $\mathbf{w}$ already live on the value-head axis.

The log-decay is computed outside the kernel in fp32,

$$\mathbf{g}_t = -\exp(\mathbf{a}) \odot \text{softplus}(\text{Proj}_f(\mathbf{x}_t) + \delta). \quad (86)$$

The vector $\mathbf{a}$ is stored per key head and broadcast across the d_k channels of that head. The bias δ is stored per key channel. The kernel consumes $\mathbf{g}_t$ directly and forms the local cumulative sums in Eq. 30.

If negative eigenvalues are enabled, only the erase gate is scaled by 2. This changes $\mathbf{b}_t \in [0, 1]^{d_k}$ into $\mathbf{b}_t \in [0, 2]^{d_k}$. The write gate remains in $[0, 1]^{d_v}$.

C.2. Forward kernels

The chunk size is fixed to $C = 64$. Each chunk is processed by the following steps.

Intra-chunk products The first kernel forms the causal score matrix $\mathbf{A}_{qk}$ and the strictly lower matrix $\mathbf{T}$. The Gated DeltaNet-2 specific computation is the row factor of $\mathbf{T}$,

$$T_{rs} = \bar{\mathbf{e}}_r^\top \bar{\mathbf{k}}_s = (\gamma_r \odot \mathbf{b}_r \odot \mathbf{k}_r)^\top (\gamma_s^{-1} \odot \mathbf{k}_s) \quad s < r. \quad (87)$$

Thus the erase gate is multiplied into the key tile before the dot product. The score matrix $\mathbf{A}_{qk}$ is unchanged apart from the same decay-normalized key factors.

WY solve The second kernel solves $\mathbf{A} = (\mathbf{I} + \mathbf{T})^{-1}$ by forward substitution. It then exposes the same lower triangular inverse to both right hand sides. This is the compact WY step used in Eq. 34.

Auxiliary construction The third kernel builds

$$\mathbf{U} = \mathbf{A}(\mathbf{W} \odot \mathbf{V}), \quad \mathbf{Y} = \mathbf{A}\bar{\mathbf{E}}. \quad (88)$$

The implementation stores $\mathbf{Y}$ using the historical buffer name w because KDA used the same buffer for its erase-side auxiliary; this buffer is not the write-gate matrix $\mathbf{W}$. The mathematical role is $\mathbf{Y}$ throughout this paper.

State and output The inter-chunk state recurrence consumes $\mathbf{K}_{\text{tail}}$, $\mathbf{Y}$, and $\mathbf{U}$, and applies Eq. 40. The output kernel consumes $\mathbf{Q}_\gamma$, $\mathbf{A}_{qk}$, and $\mathbf{R} = \mathbf{U} - \mathbf{Y}\mathbf{S}_0$, and applies Eq. 44. These two kernels do not depend on how the gate factors were produced, so they share the same matrix shapes as KDA.

C.3. Backward kernels

The backward pass mirrors Appendix B.

Output vector-Jacobian product The first backward kernel computes $d\mathbf{A}_{qk}$ and the output-path contribution to $d\mathbf{R}$ from Eq. 53 and Eq. 54. It is structure-equivalent to the KDA output vector-Jacobian product because it only sees $\mathbf{R}$.

State vector-Jacobian product The second backward kernel propagates $d\mathbf{S}_C$ through Eq. 51. It is also structure-equivalent to the KDA state vector-Jacobian product because it consumes $\mathbf{K}_{\text{tail}}$, $\mathbf{Y}$, and $\mathbf{U}$ as already formed tensors.

Gate-aware WY vector-Jacobian product The third backward kernel implements Eq. 64, Eq. 65, and Eq. 66. This is the main Gated DeltaNet-2 specific kernel. It accumulates $d\mathbf{A}$ with $\mathbf{Z}^\top = (\mathbf{W} \odot \mathbf{V})^\top$ and $\bar{\mathbf{E}}^\top = (\gamma \odot \mathbf{B} \odot \mathbf{K})^\top$. It also emits the direct gradients

$$d\mathbf{W} = d\mathbf{Z} \odot \mathbf{V}, \quad d\mathbf{V} = d\mathbf{Z} \odot \mathbf{W}, \quad d\mathbf{B} += d\bar{\mathbf{E}} \odot \gamma \odot \mathbf{K}. \quad (89)$$

The erase-gate gradient has shape $B \times T \times H \times d_k$. The write-gate gradient has shape $B \times T \times H_v \times d_v$.

Intra-chunk vector-Jacobian product The fourth backward kernel propagates through $\mathbf{A}_{qk}$ and $\mathbf{T}$. It adds the remaining contributions to $d\mathbf{Q}$, $d\mathbf{K}$, $d\mathbf{B}$, and $d\mathbf{g}$. The dependence on the cumulative decay is reduced by a reverse cumulative sum, as in Eq. 82.

C.4. Autotuning and hardware dispatch

The fused WY backward kernel uses the same matrix shapes as the forward solve, but it has a denser set of live accumulators because it emits gradients for $\mathbf{B}$ and $\mathbf{W}$. On Hopper GPUs we restrict the warp search for this kernel to two and four warps, since the eight-warp schedule can trigger a Triton WGMMMA layout assertion for the 64×64 accumulator. On Ampere GPUs the full search space is retained. This restriction changes only the schedule, not the mathematical operation.

C.5. Recurrent decoding kernel

A forward-only recurrent kernel is provided for autoregressive decoding at short sequence lengths. It applies Eq. 29 token by token. The kernel keeps the state in fp32, multiplies it by $\exp(\mathbf{g}_t)$, reads the decayed state along $\mathbf{b}_t \odot \mathbf{k}_t$, writes $\mathbf{w}_t \odot \mathbf{v}_t$ along $\mathbf{k}_t$, and returns $\mathbf{S}_t^\top \mathbf{q}_t$. Training uses the chunk kernel.

C.6. Variable-length sequences

Packed variable-length batches are represented with cumulative sequence lengths. The chunk index construction resets the recurrent state at every sequence boundary. The same layout is used by the chunk forward, the chunk backward, and the recurrent decoding kernel. Padding is removed before the layer and restored after the output projection.

D. Numerical details and verification

D.1. Decay precision

The decay gate in Eq. 86 is computed in explicit fp32 before entering the kernels. This is important because the local cumulative sum $G_r = \sum_{i \leq r} g_i$ is a path-length-dependent quantity. A low precision mantissa can perturb long products of decays even when each tokenwise gate is small. The kernels therefore receive the log-decay tensor and only compute local cumulative sums and exponentials.

D.2. Query and key normalization

Queries and keys are L2-normalized per head before the recurrent update. With normalized keys, $k_t k_t^\top$ is a projector in the tied-gate limit. In Gated DeltaNet-2, the erase factor is asymmetric, but normalization still stabilizes the scale of both A_{qk} and T . The backward applies the standard L2-normalization vector-Jacobian product.

D.3. State and accumulator dtypes

The recurrent state is stored in fp32 across chunks and during recurrent decoding. Matrix multiplication accumulators use fp32. The layer output is cast back to the model dtype at the kernel boundary. The WY auxiliaries may be stored in the model dtype after fp32 accumulation, since they are recomputed when the memory-saving training path is used.

D.4. WY solve precision

The triangular solve for $A = (I + T)^{-1}$ is the most precision-sensitive part of the chunk computation. Errors in this solve are propagated through dependent forward-substitution steps. The implementation therefore exposes an explicit precision flag for the solve. The conservative IEEE fp32 path is used when required by the hardware check, while the remaining matrix products can use the faster tensor-core path.

D.5. Initialization and output gate

All linear layers are initialized with Xavier uniform weights and gain $2^{-2.5}$. Biases are initialized to zero when present. After the attention computation, the output is passed through an RMSNorm and SiLU gate before the final output projection. These choices match the training recipe used for the Gated DeltaNet family and keep the early recurrent state magnitudes controlled.

D.6. Correctness checks

We verified the chunkwise forward against a tokenwise recurrent reference for random configurations covering different sequence lengths, head counts, key dimensions, value dimensions, initial states, packed layouts, and dtypes. We verified the backward against autograd through the recurrent reference. In fp64 reference tests, gradients for Q , K , V , B , W , the log-decay, and the initial state agree to machine precision. In production fp32, differences are at the expected tensor-core accumulation noise level. In bfloat16, the error follows the bfloat16 mantissa.

E. Experimental settings

E.1. Training

We evaluate Gated DeltaNet-2 against a Transformer baseline and recent recurrent architectures, including Mamba-2 [8], Gated DeltaNet [11], Kimi Delta Attention (KDA) [12], and Mamba-3 [13]. For each recurrent architecture, we train a recurrent-only model and a hybrid model. The hybrid model follows Section 3.5, using the same recurrent token mixer together with sliding-window attention (SWA) under the same residual block structure. For Mamba-3, we evaluate both SISO and MIMO variants. The Mamba-3 MIMO model uses rank $R = 4$.

For fair recurrent comparisons, we match both parameter count and main recurrent state size. Gated DeltaNet, KDA, and Gated DeltaNet-2 use $H = 16$ heads with $d_k = 128$ and $d_v = 128$, giving a per-layer recurrent state of

$$H d_k d_v = 16 \cdot 128 \cdot 128 = 262,144 \quad (90)$$

floats per batch element. Since $d_{\text{model}} = 2048$, this equals $128d_{\text{model}}$. For Mamba-2 and Mamba-3, we use expansion factor 2 and head dimension 64, and set $d_{\text{state}} = 64$. Their main recurrent state size is therefore

$$(2d_{\text{model}})d_{\text{state}} = 4096 \cdot 64 = 262,144. \quad (91)$$

Mamba-3 MIMO keeps the same main recurrent state size as the SISO variant while adding the rank- R MIMO parameterization.

Unless stated otherwise, all models have 1.3B parameters and are trained on 100B tokens sampled from FineWeb-Edu [26]. We use AdamW with peak learning rate 4×10^{-4} , weight decay 0.1, and gradient clipping at 1.0. The learning rate follows cosine annealing with a 1B-token warm-up. The global batch size is 0.5M tokens. The training sequence length is 4K tokens. Hybrid models use a 2K SWA window.

E.2. Evaluation

Language modeling and common-sense reasoning We use the evaluation suite commonly adopted for pretrained recurrent language models [41]. Language modeling quality is measured by perplexity on WikiText [Wiki. 27] and LAMBADA [LMB. 28]. For zero-shot transfer, we report LAMBADA accuracy together with PIQA [29], HellaSwag [Hella. 30], WinoGrande [Wino. 31], ARC-Easy and ARC-Challenge [ARC-e and ARC-c 32], OpenBookQA [OBQA 33], Social IQa [SIQA 34], and BoolQ [35]. This mix covers next-token prediction, physical and social reasoning, commonsense completion, and elementary science QA.

In-context retrieval We evaluate retrieval in both controlled synthetic settings and real-data settings. For synthetic retrieval, we use Single Needle-In-A-Haystack (S-NIAH) and Multi-Key Needle-In-A-Haystack (MK-NIAH) tasks from RULER [36]. The S-NIAH suite contains three progressively harder cases. S-NIAH-1 is passkey retrieval, S-NIAH-2 asks for a numerical needle, and S-NIAH-3 asks for a word-based needle. We additionally evaluate MK-NIAH-1 where several distractor key-value pairs are present and the model must return the value associated with one requested key.

For real-world retrieval, we follow [37]. The suite includes SWDE [46] for structured relation extraction from HTML, FDA [47] for key-value retrieval from PDFs, and question-answering datasets including SQuAD [48], TriviaQA [49], DROP [50], and Natural Questions [51].

References

- [1] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020.
- [2] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR, 2021.
- [3] Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [4] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 1763–1840. PMLR, 2024.
- [5] Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. Repeat after me: Transformers are better than state space models at copying. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 21502–21521. PMLR, 2024.
- [6] Kaiyue Wen, Xingyu Dang, and Kaifeng Lyu. Rnns are not transformers (yet): The key bottleneck on in-context retrieval. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [7] Ekin Akyürek, Bailin Wang, Yoon Kim, and Jacob Andreas. In-context language learning: Architectures and algorithms. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 787–812. PMLR, 2024.
- [8] Tri Dao and Albert Gu. Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR, 2024.
- [9] Bernard Widrow, Marcian E Hoff, et al. Adaptive switching circuits. In *IRE WESCON convention record*, volume 4, pages 96–104. New York, 1960.
- [10] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems 37*, pages 115491–115522, 2024.
- [11] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [12] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiayi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.
- [13] Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [14] Christian H. Bischof and Charles Van Loan. The WY representation for products of householder matrices. In *SIAM Conference on Parallel Processing for Scientific Computing*, 1985.
- [15] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc V. Le. Transformer quality in linear time. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 9099–9117. PMLR, 2022.
- [16] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *ArXiv preprint*, abs/2307.08621, 2023.
- [17] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 56501–56523. PMLR, 2024.

- [18] Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. The dual form of neural networks revisited: Connecting test time predictions to training patterns via spotlights of attention. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 9639–9659. PMLR, 2022.
- [19] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522. PMLR, 2025.
- [20] Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [21] Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. Longhorn: State space models are amortized online learners. In *International Conference on Learning Representations*, volume 2025, pages 95419–95434, 2025.
- [22] Thierry Joffrain, Tze Meng Low, Enrique S. Quintana-Ortí, Robert A. van de Geijn, and Field G. Van Zee. Accumulating householder transformations, revisited. *ACM Trans. Math. Softw.*, 32:169–179, 2006.
- [23] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, 2019.
- [24] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models, 2024.
- [25] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [26] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *ArXiv preprint*, abs/2406.17557, 2024.
- [27] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [28] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, Berlin, Germany, 2016. Association for Computational Linguistics.
- [29] Yonatan Bisk, Rowan Zellers, Ronan LeBras, Jianfeng Gao, and Yejin Choi. PIQA: reasoning about physical commonsense in natural language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 7432–7439. AAAI Press, 2020.
- [30] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, 2019. Association for Computational Linguistics.
- [31] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 8732–8740. AAAI Press, 2020.
- [32] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv preprint*, abs/1803.05457, 2018.
- [33] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, Brussels, Belgium, 2018. Association for Computational Linguistics.

- [34] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Social IQa: Commonsense reasoning about social interactions. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4463–4473, Hong Kong, China, 2019. Association for Computational Linguistics.
- [35] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.
- [36] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *ArXiv preprint*, abs/2404.06654, 2024.
- [37] Simran Arora, Aman Timalsina, Aaryan Singhal, Benjamin Spector, Sabri Eyuboglu, Xinyi Zhao, Ashish Rao, Atri Rudra, and Christopher Ré. Just read twice: closing the recall gap for recurrent language models. In *Proceedings of the 2nd Efficient Systems for Foundation Models Workshop at the International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- [38] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [39] Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. Simplified state space layers for sequence modeling. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [40] Antonio Orvieto, Samuel L. Smith, Albert Gu, Anushan Fernando, Çağlar Gülçehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 26670–26698. PMLR, 2023.
- [41] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. 2023.
- [42] Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, Dong Li, Weigao Sun, and Yiran Zhong. Hgrn2: Gated linear rnns with state expansion. *ArXiv preprint*, abs/2404.07904, 2024.
- [43] E. Gardner. The space of interactions in neural network models. *Journal of Physics A*, 21:257–270, 1988.
- [44] DL Prados and SC Kak. Neural network capacity using delta rule. *Electronics Letters*, 3(25):197–199, 1989.
- [45] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 7703–7717, 2021.
- [46] Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. OpenCeres: When open information extraction meets the semi-structured web. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3047–3056, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.
- [47] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes, 2023.
- [48] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for SQuAD. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia, 2018. Association for Computational Linguistics.
- [49] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, 2017. Association for Computational Linguistics.

- [50] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2368–2378, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.
- [51] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.